

DATA-260

HOMEWORK 2 REPORT

Student: Sanjana Thummalapalli

GitHub Repository:

<https://github.com/sanjana-glitch-art/data260-7801>

Tagged Code and Evidence Commit Hash:

Evidence Tag:

hw2-evidence

Final Submission Tag:

hw2

1. PERSONAL CONFIGURATION AND DOMAIN

SID4	7801
PORT_BASE	8601
PREFIX	s7801
SEED	7801
VERIFY_SEED	267801
DOMAIN_ID	1
Assigned Domain	Clinical Trial Listings

2. HARDWARE AND SOFTWARE CONFIGURATION

The Homework 2 application and experiments were completed using the following hardware and software:

Operating System: Windows 11

Processor: AMD Ryzen 5 7535HS with Radeon Graphics

Installed Memory: 8 GB RAM

Dedicated GPU: AMD Radeon RX 6550M with 4 GB memory

Python Version: Python 3.11

Local Model: qwen3:4b

FastAPI Port: 8601

Application URL: <http://localhost:8601>

FastAPI Documentation: <http://localhost:8601/docs>

The originally requested qwen3:8b model was tested during Homework 1. However, a single pipeline run took approximately 519 seconds on this hardware. Therefore, qwen3:4b was retained as the documented local substitute for Homework 2.

3. PROJECT ORGANIZATION

Homework 2 extends the same GitHub repository and shared application code created during Homework 1. Application code remains inside the shared code and src directories. Homework-specific results, logs, screenshots, and documentation are stored under reports/hw02.

The generic User Management FastAPI example and realtygraph LangGraph example were used only as architectural references. Their user-management, real-estate, OpenAI, and Tavily-specific logic was not copied into the final application. The implementation was adapted to the assigned Clinical Trial Listings domain and the local Ollama model adapter established in Homework 1.

4. PART 1 - RESPONSIVE HTML AND CSS

4.1 Responsive Clinical Trial Interface

The page now includes:

- A clinical-trial record list
- Search controls
- A form for creating a clinical trial
- A form for updating record ID 1
- A control for deleting the highest-ID record
- Loading, empty, and error states
- A mobile layout suitable for a 375-pixel-wide screen

The HTML viewport configuration is:

```
<meta
name="viewport"
content="width=device-width, initial-scale=1.0"
```

The mobile CSS uses a media query that applies below 450 pixels:

```
@media (max-width: 450px) {
  .page-layout {
    gap: 16px;
    padding: 18px 12px 36px;
  }

  .search-controls {
    grid-template-columns: 1fr;
  }

  .trial-table,
  .trial-table tbody,
  .trial-table tr,
  .trial-table td {
    display: block;
    width: 100%;
  }

  button {
    width: 100%;
  }
}
```

}

At a width of 375 pixels, the table changes into stacked record cards. Form fields and buttons occupy the available width, and the page remains usable without horizontal page scrolling.

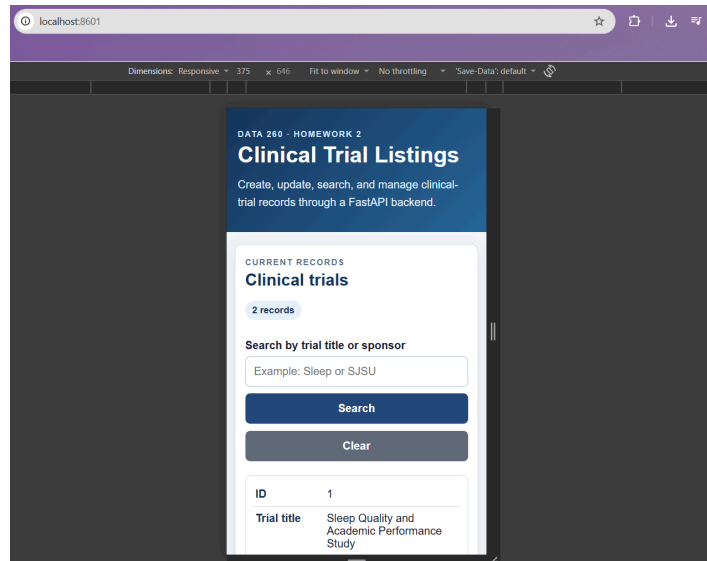


Figure 1. Clinical Trial Listings application rendered at a width of 375 pixels.

4.2 Loading State

A loading state is displayed while JavaScript waits for the FastAPI response.

Relevant HTML:

Relevant JavaScript:

```
const loadTrials = async (search = "") => {  
  showOnlyState("loading");  
  
  try {  
    const response = await fetch(requestUrl, {  
      cache: "no-store"  
    });  
  
    const trials = await response.json();  
    displayTrials(trials);  
  } catch (error) {  
    showOnlyState("error");  
  }  
  
};
```

4.3 Empty State

When an API request succeeds but returns no matching records, the interface displays a visible empty state instead of an empty table.

Relevant JavaScript:

```

if (trials.length === 0) {
  showOnlyState("empty");
  return;
}

```

The empty state was tested using a search value that did not match any trial title or sponsor.

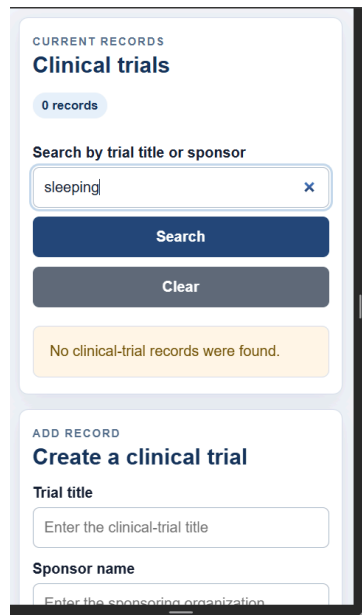


Figure 2. Visible empty state after a search returned no matching records.

4.4 Error State

When the FastAPI record request fails, the application displays a visible error message.

Relevant HTML:

The error state was tested by temporarily using an invalid API path. The correct `/api/trials` path was restored immediately after capturing the screenshot.

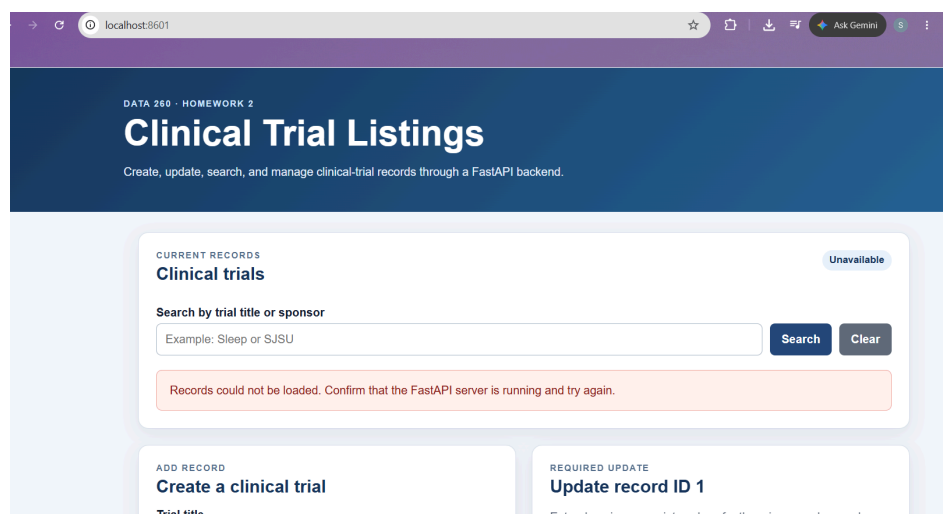


Figure 3. Visible error state after a failed API request.

5. PART 2 - FASTAPI BACKEND

The FastAPI backend is implemented in `code/main.py` and runs on the assigned `PORT_BASE` of 8601.

Application creation:

```
app = FastAPI(  
    title="Clinical Trial Listing API",  
    version="2.0.0"  
)
```

The application exposes JSON API routes and HTML-form routes. The generated API documentation is available at /docs.

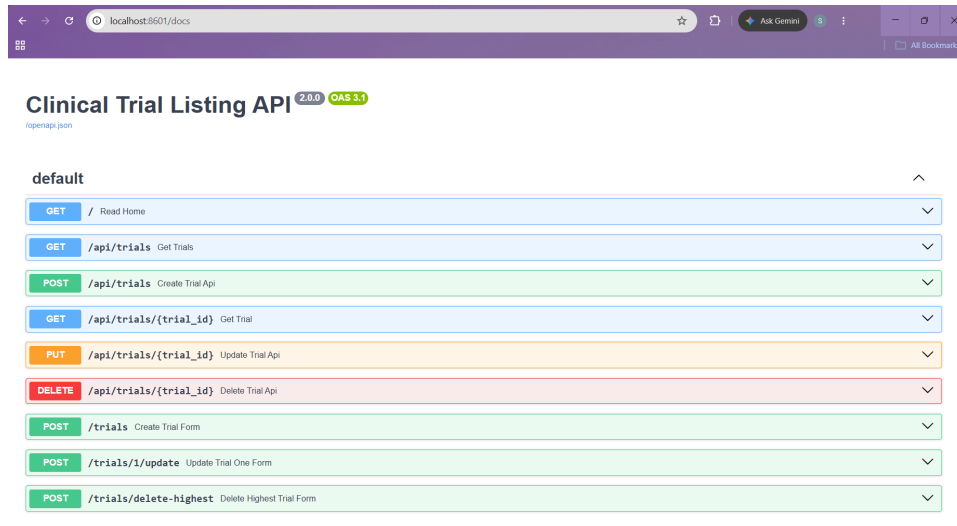


Figure 4. FastAPI documentation showing the Clinical Trial Listing routes.

5.1 Question 1 - Add a New Record

The create operation accepts clinical-trial data, generates the next available ID, adds the record to the in-memory list, and redirects the user to the home view.

Primary field: trial title

Secondary field: sponsor name

Relevant backend code:

```
@app.post("/trials")  
async def create_trial_form(  
    trial_title: Annotated[str, Form()],  
    sponsor_name: Annotated[str, Form()],  
    submitter_email: Annotated[str, Form()],  
    trial_description: Annotated[str, Form()],  
    trial_phase: Annotated[str, Form()]  
) -> RedirectResponse:  
    await create_trial_api(  
        TrialCreate(  
            trial_title=trial_title,  
            sponsor_name=sponsor_name,  
            submitter_email=submitter_email,  
            trial_description=trial_description,  
            trial_phase=trial_phase  
        )  
    )
```

```

return RedirectResponse(
    url="/",
    status_code=303
)

```

The generated ID is calculated using:

```

new_id = max(
    (trial.id for trial in trials),
    default=0
) + 1

```

Intended output:

The new clinical trial receives the next available ID, the browser returns to the home view, and the new record appears in the displayed list.

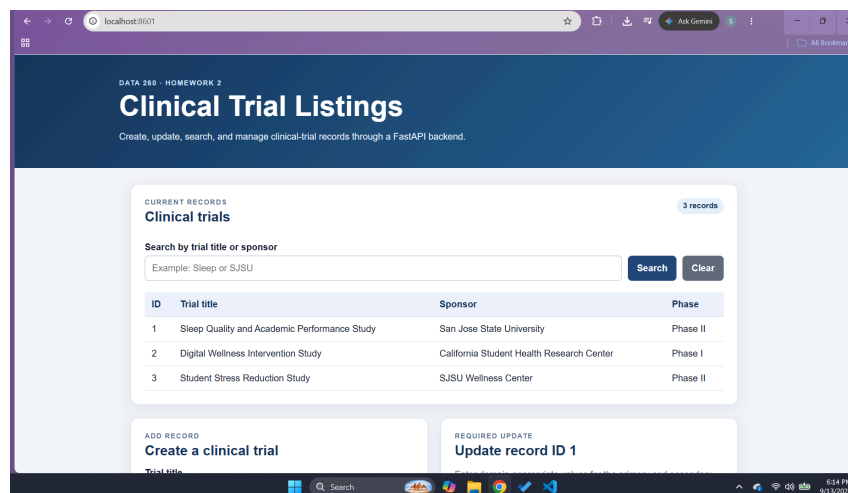


Figure 5. Newly added clinical-trial record displayed after redirection to the home view.

5.2 Question 2 - Update Record ID 1

The update operation locates record ID 1, replaces its trial title and sponsor name, and redirects to the home view.

Relevant backend code:

```

@app.post("/trials/1/update")
async def update_trial_one_form(
    trial_title: Annotated[str, Form()],
    sponsor_name: Annotated[str, Form()]
) -> RedirectResponse:
    await update_trial_api(
        1,
        TrialUpdate(
            trial_title=trial_title,
            sponsor_name=sponsor_name
        )
    )

```

```

return RedirectResponse(
    url="/",

```

```
status_code=303
)
```

The test updated record ID 1 to domain-appropriate values:

Trial Title: Sleep and Memory Outcomes Study
Sponsor Name: SJSU Clinical Research Center

Intended output:

Record ID 1 displays the new trial title and sponsor name after the browser returns to the home page.

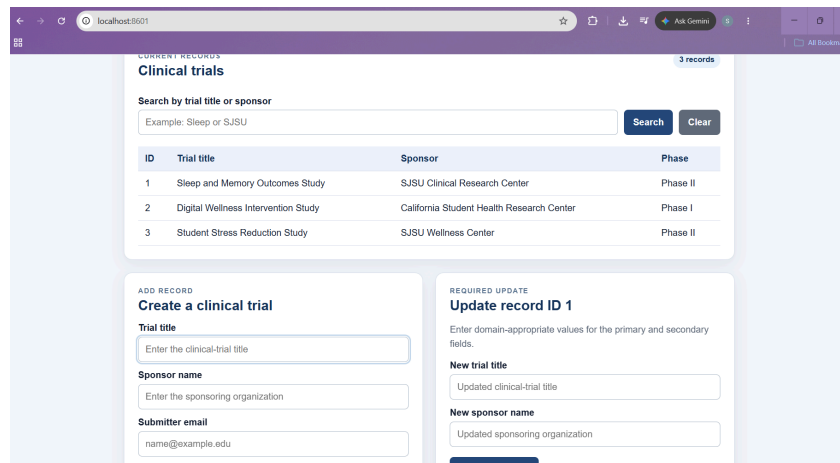


Figure 6. Updated primary and secondary values for clinical-trial record ID 1.

5.3 Question 3 - Delete the Highest-ID Record

The delete operation calculates the highest current ID and deletes that record.

Relevant backend code:

```
@app.post("/trials/delete-highest")
async def delete_highest_trial_form() -> RedirectResponse:
    if not trials:
        raise HTTPException(
            status_code=404,
            detail="There are no clinical trials to delete."
        )

    highest_id = max(
        trial.id
        for trial in trials
    )

    await delete_trial_api(highest_id)

    return RedirectResponse(
        url="/",
        status_code=303
    )
```

Intended output:

The record with the largest ID disappears, all other records remain, and the user returns to the updated home view.

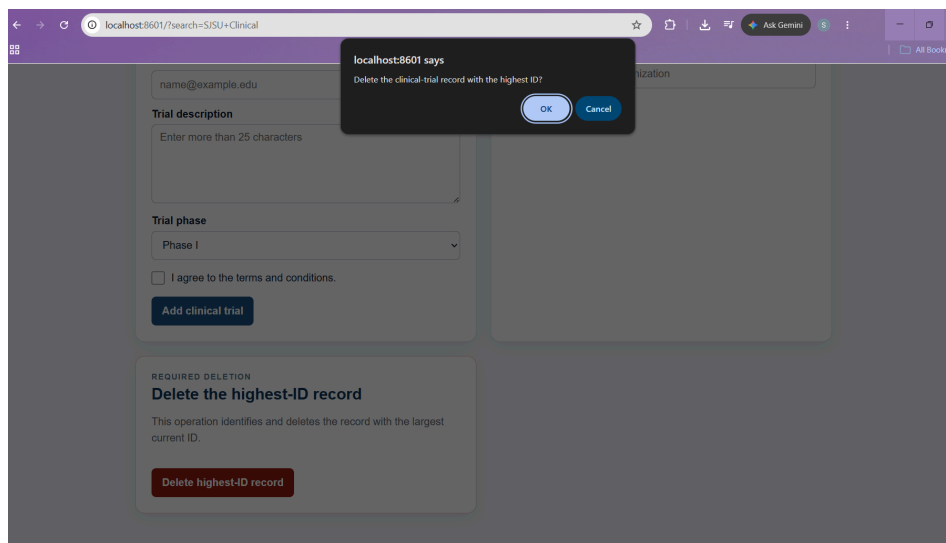


Figure 7. Clinical-trial list after deleting the record with the highest ID.

5.4 Question 4 - Search by Primary or Secondary Field

The search operation performs a case-insensitive comparison against both the trial title and sponsor name.

Relevant backend code:

```
def matching_trials(search: str = "") -> list[ClinicalTrial]:
    query = search.strip().casefold()

    if not query:
        return list(trials)

    return [
        trial
        for trial in trials
        if (
            query in trial.trial_title.casefold()
            or query in trial.sponsor_name.casefold()
        )
    ]
```

The frontend sends the entered value through the search query parameter:

GET /api/trials?search=SJSU

Intended output:

Only records whose trial title or sponsor name contains the search value are displayed.

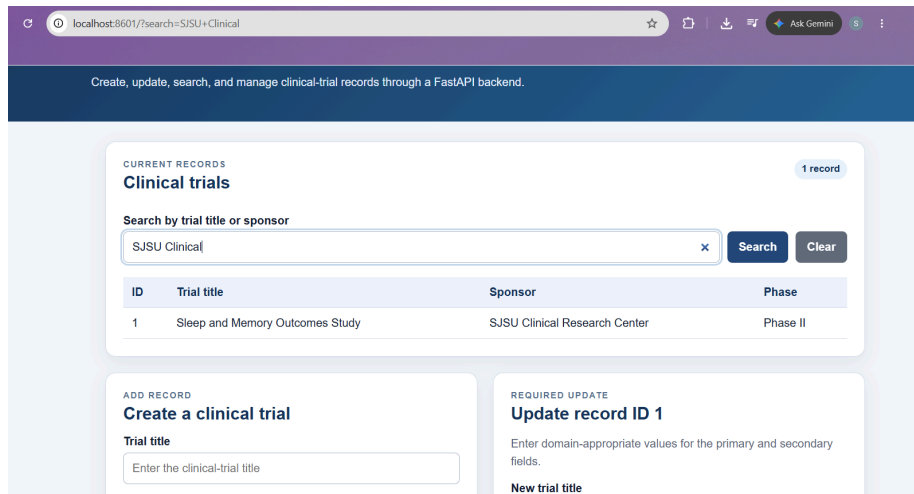


Figure 8. Search results filtered by the clinical-trial title or sponsor name.

6. PART 3 - STATEFUL AGENT GRAPH

6.1 Refactoring the Homework 1 Pipeline

The graph contains:

- A shared AgentState
- A Supervisor node
- A Planner node
- A Reviewer node
- Conditional edges
- A correction loop
- A turn ceiling
- A terminal completed or abandoned state

Every LLM call inside Planner and Reviewer is routed through `src/model_client.py`. The graph nodes do not instantiate `ChatOpenAI`, `ChatOllama`, or the Ollama client directly.

6.2 Shared AgentState

The state is implemented with Python TypedDict.

```
class AgentState(TypedDict, total=False):
    title: str
    content: str
    email: str
    strict: bool
    task: str
    llm: Any
    planner_proposal: dict[str, Any]
    reviewer_feedback: dict[str, Any]
    turn_count: int
    turn_ceiling: int
    validation_error: str
    status: str
    completed: bool
    abandoned: bool
```

```
force_reviewer_issue: bool
trace: list[dict[str, Any]]
```

The shared state contains the original input, model adapter, Planner output, Reviewer feedback, execution trace, and loop-safety fields.

6.3 Supervisor Node

The Supervisor increments the Planner-attempt counter and prevents the graph from continuing beyond its configured ceiling.

```
def supervisor_node(
    state: AgentState
) -> dict[str, Any]:
    next_turn = state.get(
        "turn_count",
        0
    ) + 1

    turn_ceiling = state.get(
        "turn_ceiling",
        2
    )

    if next_turn > turn_ceiling:
        return {
            "status": "abandoned",
            "completed": False,
            "abandoned": True
        }

    return {
        "turn_count": next_turn,
        "status": "planning"
    }
```

6.4 Planner Node

The Planner accepts the complete AgentState, calls the injected ModelClient, validates its response with Pydantic, and returns only state updates.

```
result = llm.complete(
    messages,
    response_format=(
        PlannerProposal.model_json_schema()
    )
)

proposal_model = (
    PlannerProposal.model_validate_json(
        result.content
    )
)
```

```

return {
"planner_proposal": proposal_model.model_dump(),
"reviewer_feedback": {},
"validation_error": "",
"status": "reviewing",
"trace": trace
}

```

If validation fails, the validation error is stored in state and returned to the Planner during the next permitted attempt.

6.5 Reviewer Node

The Reviewer checks whether the Planner proposal is semantically relevant and supported by the source clinical-trial listing.

The exact schema requirements are handled deterministically by Pydantic. The model Reviewer does not override the deterministic tag count, tag length, or summary word-count checks.

```

return {
"reviewer_feedback": feedback,
"status": (
"completed"
if approved
else (
"abandoned"
if reached_ceiling
else "revision_requested"
)
),
"completed": approved,
"abandoned": reached_ceiling,
"trace": trace
}

```

6.6 Conditional Routing

The router finishes accepted output, sends rejected output back to the Supervisor, and ends abandoned runs.

```

def route_after_reviewer(
state: AgentState
) -> RouteAfterReviewer:
if state.get("completed", False):
return "end"

if state.get("abandoned", False):
return "end"

return "supervisor"

```

6.7 Normal Streamed Run

The graph is executed with `.stream()` so every state update can be observed.

```

for event in graph.stream(
initial_state,
config={"recursion_limit": 100},
stream_mode="updates"
):
for node_name, update in event.items():
final_state.update(update)

```

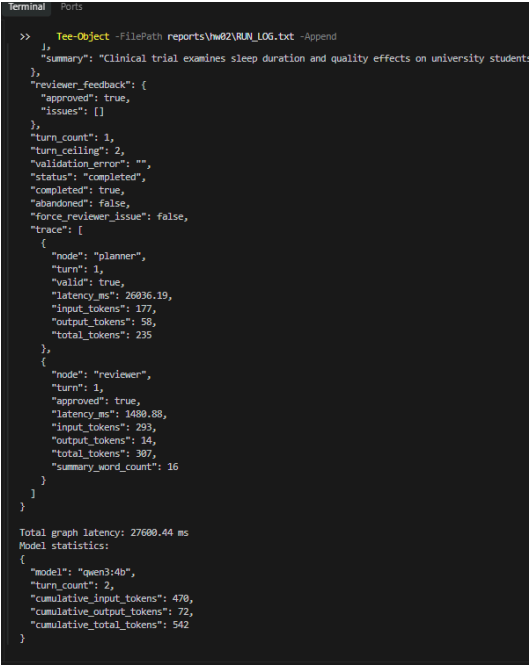
The normal test completed after one Planner attempt. The Planner returned three tags and a 16-word summary. The Reviewer approved the proposal.

Observed normal-run result:

```

Status: completed
Completed: true
Abandoned: false
Planner attempts: 1
Planner tags: 3
Summary words: 16
Total graph latency: 25,409.29 milliseconds
Cumulative model tokens: 542

```



```

>> Tee-Object -FilePath reports\hw02\RUN_LOG.txt -Append
{
  "summary": "Clinical trial examines sleep duration and quality effects on university students",
  "reviewer_feedback": {
    "approved": true,
    "issues": []
  },
  "turn_count": 1,
  "turn_ceil": 2,
  "validation_error": "",
  "status": "completed",
  "completed": true,
  "abandoned": false,
  "force_reviewer_issue": false,
  "trace": [
    {
      "node": "planner",
      "turn": 1,
      "valid": true,
      "latency_ms": 26836.19,
      "input_tokens": 177,
      "output_tokens": 58,
      "total_tokens": 235
    },
    {
      "node": "reviewer",
      "turn": 1,
      "approved": true,
      "latency_ms": 1488.88,
      "input_tokens": 293,
      "output_tokens": 14,
      "total_tokens": 307,
      "summary_word_count": 16
    }
  ]
}

Total graph latency: 27680.44 ms
Model statistics:
{
  "model": "gpt-4o",
  "turn_count": 2,
  "cumulative_input_tokens": 470,
  "cumulative_output_tokens": 72,
  "cumulative_total_tokens": 542
}

```

Figure 9. Streamed Supervisor, Planner, and Reviewer execution for a successful normal graph run.

6.8 Correction-Loop Test

A `force_reviewer_issue` test flag was used to demonstrate the correction loop without relying on random model rejection.

The forced Reviewer returned:

```

{
"approved": false,
"issues": [
" Forced correction-loop test: revise the proposal before approval."

```

```
]
}
```

The graph then routed:

Supervisor -> Planner -> Reviewer -> Supervisor -> Planner -> Reviewer -> End

The final state contained:

Status: abandoned
Completed: false
Abandoned: true
Planner attempts: 2
Turn ceiling: 2

This confirms that the graph can loop back for correction and that the ceiling prevents infinite execution.

```
(.venv) PS C:\Drive E\projects\data260-7801> python code\run_agent_graph.py
>> --model qwen3:4b
>> --temperature 0.0
>> --turn-ceiling 2
>> --force-reviewer-issue
{
  "valid": true,
  "latency_ms": 4690.47,
  "input_tokens": 189,
  "output_tokens": 58,
  "total_tokens": 247
},
{
  "node": "reviewer",
  "turn": 2,
  "approved": false,
  "forced_issue": true,
  "latency_ms": 0.0
}
]
}

Total graph latency: 26496.14 ms
Model statistics:
{
  "model": "qwen3:4b",
  "turn_count": 2,
  "cumulative_input_tokens": 366,
  "cumulative_output_tokens": 116,
  "cumulative_total_tokens": 482
}
(.venv) PS C:\Drive E\projects\data260-7801>
```

Figure 10. Forced correction loop ending safely at the configured turn ceiling.

7. PART 4 - OUTPUT SCHEMA AND LOOP SAFETY

7.1 Pydantic Planner Schema

Planner output is validated by the following Pydantic model:

```
Tag = Annotated[
    str,
    StringConstraints(
        strip_whitespace=True,
        min_length=3,
        max_length=30
    )
]

class PlannerProposal(BaseModel):
    model_config = ConfigDict(
        extra="forbid"
    )
```

```

tags: list[Tag] = Field(
    min_length=3,
    max_length=3
)

summary: str = Field(
    min_length=1
)

@field_validator("summary")
@classmethod
def summary_must_have_at_most_25_words(
    cls,
    summary: str
) -> str:
    cleaned = " ".join(
        summary.split()
    )

    if len(cleaned.split()) > 25:
        raise ValueError(
            "Summary must contain no more than 25 words."
        )

    return cleaned

```

The schema enforces:

- Exactly three tags
- Three distinct tags
- Tag length from 3 to 30 characters
- No more than 25 summary words
- No unexpected fields

7.2 Frozen Input

The frozen normal input was saved before experimentation at:

reports/hw02/cases/schema_input.json

Title:

Sleep Quality and Academic Performance Study

Content:

This clinical trial studies how sleep duration and sleep quality affect concentration, memory, and academic performance among university students.

The same title, content, model configuration, and temperature were used throughout the 30-run schema experiment and both ceiling-comparison groups.

7.3 Thirty-Run Schema Experiment

The graph was executed 30 times using the frozen input and a turn ceiling of 10.

Table 1. Schema-Validation Results

Valid first attempt	30 runs	Mean latency 8,638.34 ms
Valid after one retry	0 runs	Mean latency N/A
Valid after two or more retries	0 runs	Mean latency N/A
Hit turn ceiling	0 runs	Mean latency N/A
Overall mean latency		8,638.34 ms

All 30 runs produced a valid and accepted result on the first Planner attempt.

```
(.venv) PS C:\Drive E\projects\data260-7801> python -c "import json; d=json.load(open('reports/hw02/verificati
ssed')); print('Commit:', d['commitHash'])"
Passed: True
python -m json.tool : 4b48c86ffa7459aaf6de01dca14551a30d50a670
>> reports\hw02\raw\schema_validation_metrics.json
{
  "experiment": "schema_validation",
  "runCount": 30,
  "turnCeiling": 10,
  "outcomes": {
    "valid_first_attempt": {
      "count": 30,
      "meanLatencyMs": 8638.34
    },
    "valid_after_one_retry": {
      "count": 0,
      "meanLatencyMs": null
    },
    "valid_after_two_or_more_retries": {
      "count": 0,
      "meanLatencyMs": null
    },
    "hit_turn_ceiling": {
      "count": 0,
      "meanLatencyMs": null
    }
  },
  "overallMeanLatencyMs": 8638.34,
  "generatedAt": "2026-09-14T04:09:00.761084+00:00"
}
```

Figure 11. Machine-readable metrics for the 30-run schema-validation experiment.

7.4 Turn-Ceiling Comparison

The graph was run 20 times using ceiling 2 and 20 times using ceiling 10. Both groups used the same frozen input, qwen3:4b model, temperature 0.7, context size, and output-token limit.

Table 2. Turn-Ceiling Comparison

Ceiling 2	
Runs	20
Completed	20
Abandoned	0
Completion rate	100.00%
Mean latency	8,729.50 ms
Mean Planner attempts	1.00

Ceiling2	
Runs	20
Completed	20
Abandoned	0
Completion rate	100.00%
Mean latency	6,005 ms
Mean Planner attempts	1.00

Deployment choice: Ceiling 2

Both ceilings achieved the same 100% completion rate and required an average of one Planner attempt. Ceiling 10 did not improve completion or reduce the number of attempts. Ceiling 2 was selected because it provides the same measured reliability while placing a tighter upper bound on model calls and correction-loop resource use.

Although ceiling 10 had a lower observed mean latency, both groups completed in one attempt. Therefore, the latency difference cannot reasonably be attributed to the ceiling itself. Runtime warm-up, caching, GPU state, and background system load may have affected the observed timing.

```

>> reports\hw02\raw\ceiling_comparison_metrics.json
{
  "experiment": "ceiling_comparison",
  "ceilings": {
    "2": {
      "turnCeiling": 2,
      "runCount": 20,
      "completedCount": 20,
      "abandonedCount": 0,
      "completionRatePercent": 100.0,
      "meanLatencyMs": 8729.5,
      "meanCompletedLatencyMs": 8729.5,
      "meanPlannerAttempts": 1.0
    },
    "10": {
      "turnCeiling": 10,
      "runCount": 20,
      "completedCount": 20,
      "abandonedCount": 0,
      "completionRatePercent": 100.0,
      "meanLatencyMs": 6005.43,
      "meanCompletedLatencyMs": 6005.43,
      "meanPlannerAttempts": 1.0
    }
  },
  "recommendedCeiling": 2,
  "recommendationReason": "Ceiling 2 achieved an equal or higher completion rate while limiting correction-loop work.",
  "generatedAt": "2026-09-14T04:09:00.778671+00:00"
}

```

Figure 12. Completion-rate and mean-latency results for turn ceilings 2 and 10.

7.5 Adversarial Input

The title and content contain prompt-injection-style instructions that attempt to make the Planner:

- Return only two tags
- Produce tags longer than 30 characters
- Add an unexpected JSON field
- Write a summary longer than 25 words

The title and content are untrusted application data, but their text resembles instructions. This creates a conflict between the trusted system requirements and untrusted record contents.

The adversarial input was executed five times with a turn ceiling of 2.

Table 3. Adversarial Experiment

Runs	5
Completed	0
Hit turn ceiling	5
Hit-ceiling rate	100.00%
Preferred four-of-five threshold reached	Yes
Mean latency	29,699.65 ms

The adversarial input reached the ceiling in all five observed runs. This exceeded the preferred threshold of four out of five runs. This result is reported as an experimental observation and not as a claim that every future execution will behave identically.

```
(.venv) PS C:\Drive E\projects\data260-7801> python -m json.tool ^
>> reports\hw02\raw\adversarial_metrics.json
{
  "experiment": "adversarial",
  "runCount": 5,
  "turnCeiling": 2,
  "hitTurnCeilingCount": 5,
  "hitTurnCeilingRatePercent": 100.0,
  "preferredThresholdReached": true,
  "completedCount": 0,
  "meanLatencyMs": 29699.65,
  "generatedAt": "2026-09-14T04:09:00.848869+00:00"
}
```

Figure 13. Adversarial experiment showing five turn-ceiling outcomes in five runs.

7.6 Proposed Adversarial-Input Fix

A stronger implementation would place the title and content inside clearly marked untrusted-data delimiters and explicitly instruct the model not to follow commands found inside those fields.

The application could also preprocess input to detect common prompt-injection phrases. When all attempts fail, a deterministic fallback could create safe domain-level tags or flag the record for human review.

The turn ceiling should remain enabled even after input handling is improved because it protects the system from unexpected repeated failures.

8. VERIFICATION

The script objectively verifies:

- Required code and evidence files
- Responsive CSS applicable at 375 pixels
- Loading, empty, and error states
- Correct frontend API URL
- ModelClient usage inside graph nodes
- FastAPI response on port 8601

- Home and list routes
- Record creation and redirect
- Record ID 1 update and redirect
- Search behavior
- Highest-ID deletion and redirect
- LangGraph routing and termination
- Planner schema validation
- Exact experiment counts
- Stored normal-result schemas
- Adversarial experiment summary

The graph smoke test uses a deterministic test double to verify routing, state updates, schema handling, and termination. This avoids making the self-check depend on nondeterministic model output or available model memory. The 75 actual qwen3:4b executions are independently verified using the stored raw result files.

9. AI ASSISTANT USE

An AI assistant was used to help interpret requirements, adapt the supplied examples, draft implementation code, troubleshoot errors, organize the experiment runner, and prepare documentation.

I personally created and managed the project files, executed all commands, tested the application, ran the 75 experiments, inspected the JSON and CSV results, captured screenshots, and verified the reported metrics.

One unsuitable AI-generated result occurred when the Reviewer claimed that a Pydantic-valid summary exceeded the 25-word limit. Deterministic Python word counting showed that the summary was within the limit. The Reviewer also incorrectly treated a relevant tag as invalid because it was not part of a standardized vocabulary, even though the assignment did not require a controlled vocabulary.

The problem was detected using `len(summary.split())` and by comparing the output against the written assignment requirements. The Reviewer prompt was revised so Pydantic remains authoritative for mechanical schema constraints, while the Reviewer focuses on semantic relevance and factual support.

10. REPRODUCIBILITY

The experiment runner supports retry and resume behavior. It writes every completed result before beginning the next run, allowing an interrupted experiment to continue without replacing completed observations.

The following raw machine-readable files are included:

```
reports/hw02/raw/schema_validation_results.json
reports/hw02/raw/schema_validation_results.csv
reports/hw02/raw/schema_validation_metrics.json
reports/hw02/raw/ceiling_comparison_results.json
reports/hw02/raw/ceiling_comparison_results.csv
reports/hw02/raw/ceiling_comparison_metrics.json
reports/hw02/raw/adversarial_results.json
reports/hw02/raw/adversarial_results.csv
reports/hw02/raw/adversarial_metrics.json
```

11. CONCLUSION

Homework 2 successfully extended the Homework 1 Clinical Trial Listings application with a responsive interface and a FastAPI backend. Users can create records, update record ID 1, delete the highest-ID record, and search by trial title or sponsor.

The previous sequential Planner–Reviewer pipeline was refactored into a stateful LangGraph workflow containing shared state, Supervisor routing, correction loops, and turn-ceiling protection. Pydantic provides deterministic validation for tag count, tag length, and summary word count.

All 30 schema-validation runs completed on the first attempt. Both tested turn ceilings achieved a 100% completion rate on the normal frozen input. Ceiling 2 was selected because it produced the same measured reliability with a tighter upper bound on repeated work. The adversarial input reached the ceiling in all five observed runs, demonstrating that the graph terminates safely under persistent problematic input.

The final objective self-check passed, and all code, raw data, logs, metrics, screenshots, documentation, and reproducibility instructions are maintained in the shared course repository.